

IDENTITY V · 加页手记

《加页手记攻略》微信小程序

项目分析 · 功能清单 · 最简学习路径

一份用于快速读懂项目结构、功能边界与上手顺序的学习文档

项目名称	加页手记攻略（含主入口与学习资源入口）
项目形态	原生微信小程序（微信应用内独立运行）
数据规模	轻量级数据（仅记录学习进度与基础统计）
图表规模	基础图表（柱状图、折线图）
视觉呈现	简洁清晰的UI设计，符合学习类应用风格

目录

1. 项目概览：这是什么项目
2. 目录结构与职责
3. 分析思考：架构设计与关键机制
4. 功能清单：逐模块说明
5. 最简学习路径：从零到能维护
6. 核心代码阅读顺序
7. 发布与维护检查表
8. 总结与改进建议
9. 本轮改造记录

使用建议

第一次阅读可按第 5 节“最简学习路径”的顺序，边读边打开对应文件；熟悉后把第 7 节检查表作为日常维护基线。

1. 项目概览：这是什么项目

本项目是一个面向《第五人格》玩家自制的微信小程序查询工具，主题为“加页手记”玩法。它解决的核心问题是：在对局准备、路线复盘时，用尽量少的点击找到对应攻略图。

1.1 核心使用链路



1.2 技术栈

层次	技术选型	用途
客户端	原生微信小程序 (WXML / WXSS / JS)	页面交互、列表与详情、图片预览、分享
数据层	data/localMapIndex.js + data/api.js	单一数据源 (SSOT) 与统一数据读取接口
云端	微信云开发 (云存储 / 云函数 / 云数据库)	攻略图片临时链接、反馈图片上传、反馈入库
资源策略	分包加载 + 本地兜底分包 + cloud:// 映射	弱网可用、控制主包体积、云资源失败自动回退
工程工具	Node.js + PowerShell 脚本	素材同步压缩、云映射生成、1119 项完整性校验

1.3 关键规模数据

统计项	数值	说明
注册主包页面	11 个	首页、我的、路线查询、详情、图鉴、反馈、教程、关于等
注册分包	7 个	5 个展十版资源包 + 凉哈皮资源包 + 凉哈皮图标包
攻略路线	6 条	展十版 5 条 (新手/简单/普通/困难全棺/困难速刷) + 凉哈皮 1 条
路线形状	40 个	含 “r l t r 北 南 左 右” 等形状分组
攻略图片	123 张	按 shape/door/file 三级索引，由验证脚本统计
云映射条目	151 条	123 张攻略图 + 28 张凉哈皮识别图标
图鉴条目	54 条	inventoryData 24 条 + chapterData 30 条
完整性校验	1119 项	node tools/validate-project.js 全部通过

2. 目录结构与职责

目录可以分成四层：全局配置 → 数据/API 层 → 页面层 → 资源/工具层。先看全局，再看数据，最后看页面。

路径	职责与说明
app.js / app.json / app.wxss	应用入口、页面/分包/TabBar 注册、全局样式；app.js 初始化云开发环境。
data/localMapIndex.js	核心：单一数据源。地图、作者、路线、形状、侧门、图片文件、图鉴/辞章条目都在此。
data/api.js	唯一数据读取器。提供 getRoute、getShapes、buildImageUrl、resolveImageUrls 等接口。
data/cloudAssets.js	自动生成的 cloud:// fileID 映射，由 tools/gen-cloud-assets.js 产出。
pages/index	首页：按“地图 × 作者”生成攻略卡片，提供难度快捷入口。
pages/explorer	核心查询页：单页内完成作者、版本、形状、侧门/识别图选择。
pages/detail	攻略详情页：加载图片、失败重试、全屏预览、分享稳定链接。
pages/inventory	资料图鉴：异象 / 道具 / 辞章三分类，支持搜索与详情弹层。
pages/tutorial / feedback / mine / about	教学图、意见反馈（云函数入库）、我的、关于。
pages/version / route / door	兼容旧分享链接，redirectTo 到新的 explorer 查询页。
pkg-zhanshi-*	展十版按路线拆分的图片分包，共 5 个，每个含 assets 与 redirect 占位页。
pkg-lianghapi-v0710	凉哈皮 7.10 新版路线图片分包（28 张）。
pkg-lianghapi-icons	凉哈皮识别图标分包（28 张），与路线图文件名一一对应。
pkg-hard / pkg-normal / pkg-easy / pkg-newbie / pkg-v0710	历史遗留分包，已从 app.json 注册和打包排除；保留供迁移期参考。
cloudfunctions/addFeedback	云函数：校验反馈内容并写入 feedback 集合，自动记录 OPENID 与创建时间。
tools/validate-project.js	项目体检脚本：注册、绑定、WXML 结构、数据流、页面模拟共 1119 项检查。
tools/gen-cloud-assets.js	按本地索引与固定云前缀生成 data/cloudAssets.js（只生成映射，不上传文件）。
tools/sync-assets.ps1	从本地源盘按索引扫描素材，压缩并按分包预算写入 pkg-*/assets。
tools/import-report.json	最近一次素材同步的分包数量、体积、缺失/多余文件报告。
images / style / utils	Tab 图标、占位图、教学图与全局样式目录；工具函数（当前使用较少）。
PROJECT_PLAN.md / CLOUD_STORAGE_GUIDE.md	项目规划与云存储对接说明，是理解和维护的首选文档。

3. 分析思考：架构设计与关键机制

3.1 为什么这样设计：单一数据源 (SSOT)

所有页面的地图、作者、难度、形状、入口、图片文件名都只维护在 `data/localMapIndex.js`。页面代码不写死业务数据，而是调用 `data/api.js`。好处是新增作者或路线时，只需扩展索引和素材，不需要复制页面逻辑；坏处是对索引结构要求高，必须靠工具脚本兜底校验。

```
localMapIndex.js
├──
└──
  data/api.js
  ├──
  ├── pages/index
  ├── pages/explorer
  ├── pages/detail
  └── pages/inventory
```

3.2 云存储与本地分包的“双通道”资源策略

- 读取顺序：先用 `assetNamespace + 相对路径` 查 `cloudAssets.js` 拿到 `cloud://fileID`，再通过 `wx.cloud.getTempFileURL` 换临时 HTTPS 地址。
- 兜底回退：云映射不存在、`wx.cloud` 不可用、单条解析失败、整体请求失败时，回退到本地分包绝对路径 `/pkg-*/assets/...`。
- 性能与弱网：临时链接缓存 90 分钟（约短于 2 小时有效期）；详情页先 `loadSubpackage`，再批量解析图片链接，任一来源失败都不阻塞页面。
- 新增资源必须双写：本地分包生成一次、云端上传一次，并由 `gen-cloud-assets.js` 生成映射，三者一致才算完成。

3.3 分包策略

- 每个攻略版本一个独立分包，如 `pkg-zhanshi-hard-full`、`pkg-lianghapi-v0710`，避免一次性下载所有攻略图。
- 每个分包只注册一个 `pages/redirect/redirect` 占位页；真正下载由 `detail` 页的 `wx.loadSubpackage` 按需触发。
- 主包 `pages/explorer` 用绝对路径引用分包图片，避免相对路径在页面层级变化时失效。
- 每个分包预算约 1.85MB，由 `tools/sync-assets.ps1` 自动按 1280/1024/800/640px 与质量 70/60/50 组合寻找最接近预算的压缩档。

3.4 多作者与旧链接兼容

- 每个路线有稳定 id，旧链接通过 `legacyIds` 识别（如 `zhanshi-hard-full` 兼容旧 id “hard”）。
- 旧页面 `pages/version`、`pages/route`、`pages/door` 只做 `redirectTo` 到 `explorer`，保留分享链接可用性。
- 详情页再次分享时统一输出新 id，避免旧 id 继续扩散。
- 云目录按 `maps/{mapId}/{authorId}/{routeSlug}` 隔离，旧 `pkg-hard` 目录不复用，防止不同作者同名文件串版。

3.5 凉哈皮“逐图图标”模式

凉哈皮版使用 `entryMode: "fileIcons"`。形状下没有“侧门”概念，而是每个识别图标对应一张攻略图：

```
route.entryMode = "fileIcons"
iconPackageRoot = "pkg-lianghapi-icons"
iconNamespace = "pkg-v0710/icon"
=====
===== → file ===== → =====
```

3.6 工程化校验

`tools/validate-project.js` 不是简单 lint，而是一个“可执行的发布前验收”：它会注册每个页面、模拟 wx API、跑首页/查询页/详情页/图鉴页流程、检查每条索引图片是否存在本地兜底文件、检查 `cloud://` 映射、检查旧 id 解

析、检查作者切换是否串数据。目前 1119 项检查全部通过。

3.7 值得注意的现状与风险

观察点	现状 / 风险	建议
云环境 ID 写死	app.js 中 CLOUD_ENV 为固定字符串	发布多套环境时可改为按环境配置注入
云映射不做在线验证	gen-cloud-assets.js 只按本地文件生成 fileID	保持“先上传云端，再生成映射”的流程纪律
图鉴图标外链	inventoryData 图标指向 BWIKI 外链	弱网下已回退本地占位图；必要时可代理或本地化
源素材路径绑定本机	sourceAnchor 为 F:/d5/... 绝对路径	团队协作时统一源盘目录，或把素材源纳入版本化规则
历史分包保留	pkg-hard 等目录未注册且打包排除	确认无引用后清理，或归档到仓库外
新增作者手工步骤多	需改索引 + 同步素材 + 注册分包 + 传云 + 生成映射	按 PROJECT_PLAN.md 的 7 步流程操作

4. 功能清单：逐模块说明

模块 / 页面	关键文件	已实现功能
首页	pages/index	按“地图 × 作者”生成攻略卡片；展示各难度并可直接跳转；短标签与长标签分两行排布；下拉刷新；分享小程序；入口到教学与图鉴。
路线查询（核心）	pages/explorer	作者切换、版本横向滚动切换、形状卡片（含入口数与图片数）、侧门底部弹层、单入口自动直达、凉哈皮识别图标列表、长按图标放大预览、当前路线分享。
攻略详情	pages/detail	解析 mapId/routeId/shapeId/door/file 参数；兼容 legacyIds；预加载分包；云端临时链接 + 本地兜底；图片失败标记与点击重试；全屏预览与长按保存；生成可读摘要；分享输出新稳定 id。
资料图鉴	pages/inventory	异象/道具/辞章三类 Tab；关键词搜索；品质映射（独特蓝/奇珍紫/稀世金/华彩红）；强化异象红框；点击底部抽屉展示刷新地图、应对方式、价格、重量、耐久等详情；图片失败回退占位图。
小抄教学	pages/tutorial	展示教学图片；点击全屏预览；支持分享教学页。
意见反馈	pages/feedback	文字最多 500 字；截图最多 8 张；图片上传云存储；调用 addFeedback 云函数入库；失败或云不可用时本地草稿暂存并在下次进入恢复；提交中禁用重复提交。
我的	pages/mine	版本号展示；入口到教学/反馈/关于；分享小程序；数据来源说明。
关于项目	pages/about	项目说明、内容来源、免责声明、版本展示。
旧链接兼容	pages/version、 pages/route、 pages/door	旧版参数重定向到新 explorer，保证历史分享链接仍可打开。
分包占位页	pkg-*/pages/redirect	使各资源包满足分包注册要求，资源包可按需被 wx.loadSubpackage 下载。
数据/API 层	data/api.js	getMaps / getRoute / getShapes / getShapeDetails / buildImageUrl / getImagesForShape / resolveImageUrls / loadRoutePackage 等统一接口；图片临时链接缓存 90 分钟。
云函数	cloudfunctions/addFeed back	校验非空内容，截断 500 字，限制 8 张图，写入 feedback 集合，返回统一 code/data/message。
工具链	tools/*	validate-project.js 做 1119 项发布前检查；gen-cloud-assets.js 生成云映射；sync-assets.ps1 压缩同步素材并控制分包体积。

功能边界说明

本项目是查询/展示型工具，不包含用户登录、内容发布、评论、后台管理；路线数据通过索引文件维护，反馈数据通过云函数入库后需在云开发控制台查看。

5. 最简学习路径：从零到能维护

下面按“先宏观、再数据、再核心链路、最后工具链”的顺序推进。每个阶段都给出具体文件与验收标准。

阶段	目标	学习动作	重点文件	验收标准
0. 环境准备（约10分钟）	能打开并校验项目	用微信开发者工具打开项目；终端运行 node tools/validate-project.js；通读 PROJECT_PLAN.md。	project.config.json、app.json、PROJECT_PLANN.md、tools/validate-project.js	校验输出 Validation passed: 1119 checks；知道主包 11 页、7 个分包。
1. 读数据（约30分钟）	理解 SSOT 结构	从 maps 数组开始读；找出 6 条 route 的 id/legacyIds/packageRoot/assetNamespace/shapes/shapeDetails；对照 api.js 的字段用途。	data/localMapIndex.js、data/api.js	能画出“地图→作者→路线→形状→门→文件”的层级图；能说清一条路线图片如何拼出路径。
2. 走核心链路（约40分钟）	理解查询交互	按“首页点击难度→explorer 选形状/入口→detail 显示图片”的顺序阅读；关注参数如何层层传递。	pages/index/index.js、pages/explorer/explorer.js、pages/detail/detail.js（配合 .wxml）	能口述一次点击从 onSelect Mode 到 wx.previewImage 的完整调用链；知道 _root_、fileIcons 两个特殊分支。
3. 读资源链路（约30分钟）	理解云端/本地双通道	读 api.cloudUrl、resolveImageUrls、getLocalFallback、loadRoutePackage；读 gen-cloud-assets.js 看映射如何生成。	data/api.js、data/cloudAssets.js、tools/gen-cloud-assets.js、CLOUD_STORAGE_GUIDE.md	能解释：cloud://fileID→HTTPS 临时链接；失败时如何回退到 /pkg-*/assets。
4. 读辅助功能（约30分钟）	覆盖非核心页面	读图鉴分类与搜索、反馈草稿与云函数、旧页面重定向。	pages/inventory/inventory.js、pages/feedback/feedback.js、cloudfunctions/addFeedback/index.js、pages/version/route/door	能说出图鉴三个 Tab 的数据来源；知道反馈失败时本地草稿如何恢复。
5. 读维护工具（约40分钟）	能安全新增内容	通读 validate-project.js 的检查分类；按 CLOUD_STORAGE_GUIDE.md 新增作者流程走一遍（可只做 DryRun）。	tools/validate-project.js、tools/sync-assets.ps1、tools/gen-cloud-assets.js	能按 7 步新增一条路线而不破坏 1119 项检查；理解“先传云、再生成映射”。

最少投入路径

如果时间非常有限，只做三件事：① 读 app.json 与 data/localMapIndex.js；② 读 pages/index→pages/explorer→pages/detail；③ 运行 tools/validate-project.js。这三步足以覆盖本项目 80% 的日常理解需求。

6. 核心代码阅读顺序

6.1 推荐顺序（每步约 10–20 分钟）

序号	文件	阅读重点
1	app.json	页面注册、7 个分包、TabBar；理解什么是主包与分包。
2	PROJECT_PLAN.md	产品定位、核心路径、体验原则、多作者约定、发布检查表。
3	data/localMapIndex.js	只看结构：maps[0].authors 与 maps[0].routes；任选一条 route 读 shapeDetails。
4	data/api.js	函数清单：getRoute、getShapes、getShapeDetails、buildImageUrl、resolveImageUrls、getPackageRoot。
5	pages/index/index.js	如何把 maps+routes 转换成首页卡片；onSelectMode 如何跳转。
6	pages/explorer/explorer.js	loadExplorer → selectRoute → openShape → onDoorTap → goToDetail 的主线。
7	pages/detail/detail.js	onLoad 参数解析、图片来源选择、render 与失败重试、分享参数。
8	data/cloudAssets.js	认识生成文件格式；与 api.getCloudAsset 的对应关系。
9	tools/validate-project.js	看 6 类检查：注册、配置、绑定、WXML、数据流、页面模拟。
10	tools/gen-cloud-assets.js + sync-assets.ps1	理解“索引 → 素材 → 映射”的生成链路。
11	cloudfunctions/addFeedback/index.js	云函数入参与返回规范，云数据库写入方式。
12	pages/inventory/index.js + feedback.js	辅助页面的数据加工与容错处理。

6.2 一条主线调用链（背诵版）

```
index.onSelectMode
  ■■ navigateTo explorer?mapId=&author=&routeId=
explorer.loadExplorer
  ■■ api.getAuthorsByMapId / getRoutesByMapId
  ■■ selectRoute → openShape → onDoorTap
detail.onLoad
  ■■ api.getRoute■■■ legacyIds■
  ■■ api.getImagesForShape / getShapeRootImageUrls
  ■■ api.loadRoutePackage■wx.loadSubpackage■
  ■■ api.resolveImageUrls■cloud:// → https■■■■■ /pkg-*/assets■
  ■■ render → image preview / share
```

6.3 新增作者路线的最小改动面

- data/localMapIndex.js：登记 author，新增 route (id、authorId、packageRoot、assetNamespace、shapes、shapeDetails)。
- 运行 tools/sync-assets.ps1：生成 pkg-{routeId}/assets 与缺失检查。
- app.json：注册新分包的 pages/redirect/redirect；创建对应 4 个文件。
- 上传 pkg-*/assets 到云存储 assetNamespace 后，运行 node tools/gen-cloud-assets.js。
- 最后运行 node tools/validate-project.js 并在开发者工具与真机验证。

7. 发布与维护检查表

环节	检查项	通过标准
代码与索引	运行 node tools/validate-project.js	Validation passed: 1119 checks, 零失败
编译	微信开发者工具编译与代码质量检查	无 WXML/WXSS 编译错误
主流程	首页→详情、整图路线、图鉴搜索、详情弹层、反馈草稿	均可正常操作
真机	云图片、长按图片、全屏预览、分享落地页	真机体验正常
云函数	部署 addFeedback; 反馈提交成功	feedback 集合可写入, 草稿被清除
资源一致	本地分包、索引、cloudAssets.js 三者一致	每个文件都有云映射或合法兜底
素材同步	tools/sync-assets.ps1 后检查 import-report.json	Missing / Extra 均为空
旧链接	旧 id (如 hard、v0710) 仍能打开	解析到新 route.id, 分享输出新 id

8. 总结与改进建议

8.1 一句话总结

这是一个“数据索引驱动 + 云端/本地双通道 + 工具脚本兜底”的查询型微信小程序：页面逻辑很薄，维护重点在数据、素材与发布流程。

8.2 项目当前优点

- 查询路径短，核心链路只有“首页 → 查询页 → 详情页”三层。
- 数据与页面解耦，新增作者不复制业务逻辑。
- 容错完整：加载、空数据、参数错误、图片失败、提交失败均有独立状态。
- 自动化程度高：素材同步、云映射生成、完整性校验形成闭环。
- 多作者与旧分享链接兼容策略清晰。

8.3 后续可优化方向（按优先级）

优先级	方向	价值
P0	继续把“索引-素材-云映射”一致性检查作为发布门禁	防止图片串版与发布缺图
P1	路线增加更新时间、适用游戏版本、变更摘要	降低内容过期风险
P2	最近查看记录、形状/侧门搜索	进一步减少重复点击
P2	图鉴增加品质与刷新难度筛选	数据量增长后更快定位
P3	匿名访问统计、图片失败日志、无结果搜索记录	指导内容维护优先级
P3	反馈处理状态（待确认/处理中/已解决）	运营化反馈闭环

8.4 给新维护者的三句话

- 改数据先看 localMapIndex.js，改行为先看 api.js 和对应页面。
- 任何素材变更后，先跑 tools/validate-project.js，再考虑发布。
- 云文件没有真正上传前，不要运行 gen-cloud-assets.js。

9. 本轮改造记录

以下改造已完成并通过 `node tools/validate-project.js` 全部校验 (1119 checks)。

改造项	改动内容	效果
云配置统一	新增 <code>config/cloud.js</code> ; <code>app.js</code> 与 <code>gen-cloud-assets.js</code> 共用 <code>envId</code> 与 <code>storagePrefix</code>	避免云环境 ID 与 <code>fileId</code> 前缀漂移
素材格式一致性	路线素材统一为 <code>.jpg</code> ; <code>sync-assets.ps1</code> 自动从源盘同名 <code>.png</code> 查找并输出正确扩展名; 索引同步改名	修复 <code>.png</code> 文件名包含 JPEG 字节的问题, 分包体积回到 2MB 以内
图标降级修复	<code>getShapelconUrl</code> 增加 <code>cloudReady</code> 判断; 新增 <code>getShapelconLocalUrl</code> ; 图标图片与预览增加 <code>binderror</code> 本地回退	云不可用时凉哈皮图标直接显示本地分包图标
分包预热	<code>explorer</code> 选中路线时立即 <code>loadRoutePackage</code>	用户选择形状/入口期间后台下载图片分包, 详情页更快
临时链接请求加固	<code>getTempFileURL</code> 按 50 个一批拆分; 缓存最多保留 400 条	适配接口上限, 防止 <code>storage</code> 无限增长
校验工具增强	覆盖 <code>bindlongpress</code> 与分包页面; 新增图片扩展名/文件头一致性检查; 新增云配置与图标回退检查	校验项从 773 提升到 1119, 堵住原有盲区
文档与清理	更新 <code>CLOUD_STORAGE_GUIDE.md</code> 格式约定; 删除未使用的 <code>utils</code> 与占位图	减少误导与仓库冗余
循环冒烟修复	清理 <code>explorer</code> 单入口自动跳转时的残留弹层; 移除索引中的 0 图片空入口并在页面层防御过滤	避免展示“0 张路线图”无效选项
图鉴图标本地化	54 张 BWIKI 外链图标下载压缩到 <code>images/inventory</code> , 并同步改索引为本地路径	图鉴不再依赖外链域名, 弱网可用
反馈草稿持久化	反馈截图复制到 <code>USER_DATA_PATH</code> ; 恢复草稿时校验文件; 提交/删除时清理持久化文件	小程序重启后截图草稿仍可恢复
首页状态清理	首页 <code>onShow</code> 时清除 <code>currentMode</code> 筛选状态	返回首页不再残留上次难度筛选
云函数依赖锁定	<code>wx-server-sdk</code> 从 <code>latest</code> 锁定到 4.0.2	避免重新部署时依赖漂移
私密配置脱敏	<code>project.private.config.json</code> 加入 <code>.gitignore</code> 并停止 Git 跟踪	避免开发者本地配置进入仓库

需要人工跟进

路线 `.png`→`.jpg` 后, 云端旧 `.png` 文件不会自动迁移。请把现役分包 `assets` 重新上传到云存储, 再运行 `node tools/gen-cloud-assets.js` 并做一次真机验证。